

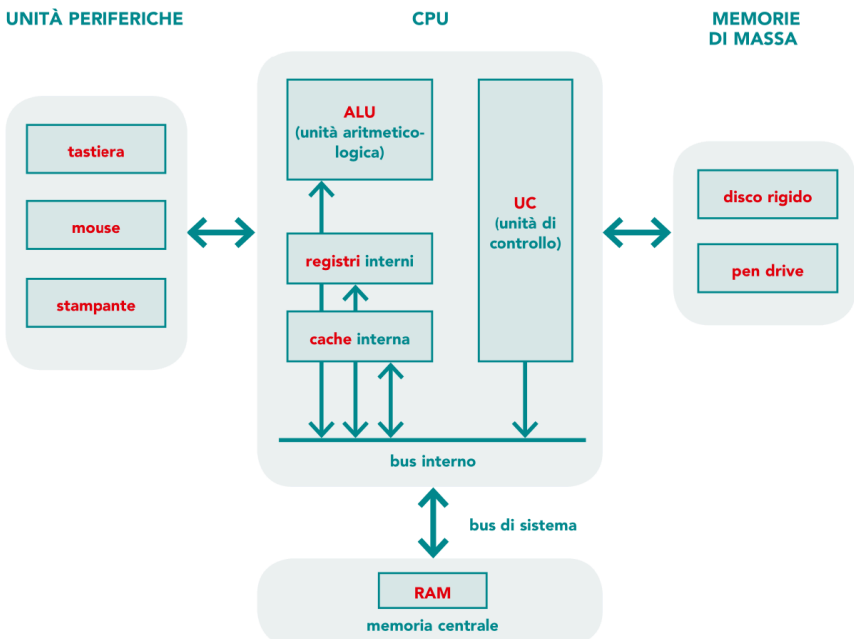
Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Architettura Essenziale di un Calcolatore

Il Modello di Von Neumann



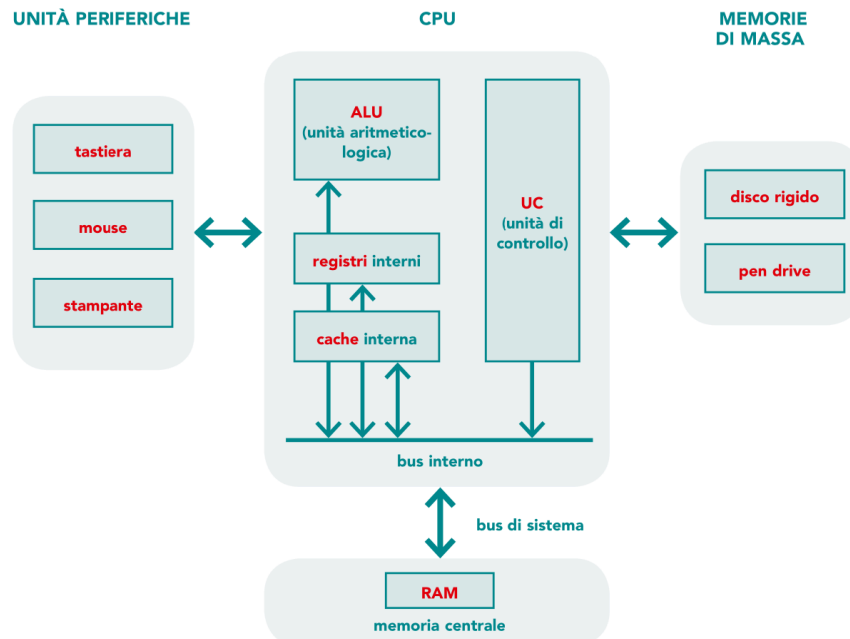
Componenti Principali

- **CPU (Central Processing Unit):** Elabora i dati ed esegue le istruzioni.
- **Memoria Principale (RAM):** Contiene dati e programmi in esecuzione.
- **Input/Output (I/O):** Interfaccia con il mondo esterno e le periferiche.
- **Bus di Sistema:** Canali di comunicazione (Dati, Indirizzi, Controllo).

La CPU: Il Cuore del Calcolatore

Unità Interne

- **Control Unit (CU):** Coordina le attività, preleva e interpreta le istruzioni.
- **ALU (Arithmetic Logic Unit):** Esegue calcoli matematici (+, -, *) e operazioni logiche (AND, OR).
- **Registri:** Memorie ultra-veloci interne alla CPU per dati temporanei.



Il Ciclo di Esecuzione (Fetch-Decode-Execute)

Le 3 Fasi del Ciclo

1. **Fetch (Prelievo):** La Control Unit legge l'istruzione dalla RAM all'indirizzo indicato dal PC.
2. **Decode (Decodifica):** La Control Unit decodifica l'istruzione e individua gli operandi.
3. **Execute (Esecuzione):** L'ALU esegue l'operazione e aggiorna registri o memoria.



Sincronizzazione tramite Clock

- Il ciclo si ripete continuamente, cadenzato dal **Clock**.
- Un clock a **3 GHz** esegue 3×10^9 cicli elementari al secondo.

Istruzioni, Assembly e Linguaggio Macchina

Livelli di Astrazione

- **Linguaggio di Alto Livello (C/Python):**

```
x = a + b;
```

- **Assembly (Mnemonico):**

```
ADD R1, R2, R3
```

- **Linguaggio Macchina (Binario):**

```
0100 0011 0001 0010
```

Anatomia di un'Istruzione Macchina

Opcode	Dest.	Op. 1	Op. 2
(Oper.)	(Dest.)	(Sorg. 1)	(Sorg. 2)
`0100`	`0011`	`0001`	`0010`
(ADD)	(Reg. R3)	(Reg. R1)	(Reg. R2)

Lo Standard ASCII

Codifica e Memoria

- Lo standard associa **7 bit** $\leftrightarrow$ un carattere.
- Per comodità, sui sistemi moderni i caratteri sono memorizzati in **8 bit (1 byte)**.
- **Caratteri Standard [0, 127]:** Sempre uguali su ogni sistema.
- **ASCII Esteso [128, 255]:** Dipende dalle opzioni di sistema.

Unicode e Linguaggi

- **UNICODE:** permette di rappresentare $\approx 10^6$ caratteri (es: ω, ぢ, 😑).
- **In Python:** Gestito automaticamente.
- **In C:** Richiede librerie esterne.
- **Nel corso:** Useremo solo i primi 128 caratteri dello standard ASCII.

Lo Standard ASCII: Esempi

Binario	Dec	Simbolo
`0000 0000`	0	`\0` (Null)
`0000 0111`	7	Bell (campanella)
`0000 1001`	9	`\t` (Tab)
`0000 1010`	10	`\n` (Fine riga)
`0010 0000`	32	Spazio
`0010 0001`	33	`!`

Binario	Dec	Simbolo
`0011 0000`	48	`'0'`
`0011 0001`	49	`'1'`
`0100 0001`	65	`'A'`
`0100 1010`	90	`'Z'`
`0110 0001`	97	`'a'`
`0111 1010`	122	`'z'`

Conversione ASCII in Python

1. `ord()` : Carattere → Codice ASCII

```python
# Da carattere a codice numerico
code_A = ord('A')
print(f"Codice ASCII di 'A': {code_A}")  # Output: 65

code_a = ord('a')
print(f"Codice ASCII di 'a': {code_a}")  # Output: 97

code_zero = ord('0')
print(f"Codice ASCII di '0': {code_zero}")  # Output: 48
```

2. `chr()` : Codice ASCII → Carattere

```python
# Da codice numerico a carattere
char_A = chr(65)
print(f"Carattere per 65: {char_A}") # Output: A

char_a = chr(97)
print(f"Carattere per 97: {char_a}") # Output: a

char_excl = chr(33)
print(f"Carattere per 33: {char_excl}") # Output: !
```

Da C a Codice Macchina (Binario)

1. Codice Sorgente C

```c
int get_number() {
    return 42;
}
```

2. Traduzione Assembly (x86-64)

```
mov eax, 42     ; Carica il numero 42 nel registro EAX
ret             ; Ritorna al chiamante
```

3. Byte Esadecimali (Opcode)

```
B8 2A 00 00 00 C3
```

- B8: Istruzione mov eax
- 2A 00 00 00: Il numero 42 (in hex $2A = 42_{10}$)
- C3: Istruzione ret

4. Codice Binario Reale salvato in memoria (0 e 1)

Istruzione Assembly	Hex	Binario Puro (8 bit per byte)
mov eax	B8	10111000
42	2A 00 00 00	00101010 00000000 00000000 00000000
ret	C3	11000011